Documentação do Projeto — Goianinho Medicina

Sep 24, 2026 · @Dhyego

Projeto da disciplina **Softwares para Dispositivos Móveis** — Engenharia de Software, Centro Universitário Alfredo Nasser (UNIFAN). Repositório: github.com/Dhyegoreis/projeto_app (privado).

1. Visão geral

Vamos construir um app Android que conecta paciente e médico por meio de um assistente de IA, o Goianinho, que organiza exames, agenda consultas e prepara tudo para o médico decidir. Nesta primeira fase entregamos só o front-end, com Firebase como backend e respostas da IA simuladas.

Problema. Entre o paciente precisar de atendimento e o médico atender bem existe um processo fragmentado: fila, exames que chegam desorganizados, agendamento manual e o médico perdendo tempo juntando informação.

Solução. Um assistente conversacional que acompanha os dois lados. O paciente fala em linguagem natural ("marca minha consulta", "manda meu exame pro médico") e o Goianinho executa. O médico recebe resumos prontos, com a fonte de cada informação, e só confirma ou corrige.

Diferencial. O mercado já tem peças soltas (triagem, agendamento, escriba de consulta). Aqui elas ficam juntas numa única conversa contínua, para paciente e médico.

Escopo desta fase. 9 telas (Login compartilhado, 4 do paciente e 4 do médico) em Kotlin + Jetpack Compose, login real via Firebase, dados reais no Firestore (sem Firebase Storage nesta fase, ver seção 8), e IA simulada com respostas fixas.

Protótipo de referência: telas no canvas. Toda tela desenvolvida deve seguir esse protótipo.

2. Princípios do produto

Toda decisão de tela ou de texto deve respeitar estas cinco regras. Na dúvida, volte aqui.

ID	Princípio	O que significa na tela
P-01	A IA apoia, o médico decide	O Goianinho nunca apresenta diagnóstico como definitivo. Todo resumo mostra "aguardando médico" até ele confirmar.
P-02	Toda informação tem fonte	Cada ponto de resumo exibe arquivo, página e linha de onde saiu.
P-03	Linguagem natural primeiro	O usuário consegue fazer tudo conversando. Botões e chips são atalhos, não o único caminho.
P-04	O Goianinho é assistente de um médico	Ela sempre se apresenta vinculada ao médico ("assistente do Dr. X").
P-05	Simples para quem não é da tecnologia	Textos curtos, botões grandes (mín. 44 px de toque) e opção de voz.

O P-01 e o P-02 atendem à Resolução CFM nº 2.454/2026, que exige que a decisão clínica seja do médico e que o uso de IA fique registrado.

3. Perfis de usuário

O app tem dois perfis, escolhidos no login. Cada um cai num fluxo próprio com quatro abas, sempre começando pelo **Início**.

Perfil	Quem é	O que faz no app	Abas	Nível técnico
Paciente	Pessoa que busca atendimento, muitas vezes sem familiaridade com tecnologia	Conversa com o Goianinho, envia exames, marca e entra na teleconsulta	Início · Assistente · Documentos · Agenda	Básico
Médico	Profissional responsável pelo atendimento	Vê o resumo do dia, revisa e confirma resumos, conduz a teleconsulta	Início · Goianinho · Revisões · Agenda	Intermediário

```
flowchart LR
  L[Login] -->|Paciente| HP[Início do paciente]
  L -->|Médico| HM[Início do médico]
  HP --- C[Chat com o Goianinho]
  HP --- D[Documentos]
  HP --- A[Agenda]
  HM --- M[Goianinho do médico]
  HM --- R[Revisões]
  HM --- AM[Agenda do dia]
```

Nos dois fluxos a barra inferior liga as quatro abas, e o Início funciona como painel de atalhos para as outras três.

4. Stack e arquitetura

Usamos a arquitetura que o próprio Google recomenda para Android: Jetpack Compose na interface, MVVM na organização e Firebase como backend. Sem backend próprio nesta fase.

Camada	Tecnologia	Uso no projeto
Linguagem	Kotlin	Todo o código
IDE	IntelliJ IDEA com plugin Android	Desenvolvimento (configuração do Firebase feita à mão, sem o Firebase Assistant)
Interface	Jetpack Compose + Material 3	Telas, componentes e tema
Organização	MVVM (Screen + ViewModel + StateFlow)	Uma ViewModel por tela
Navegação	Navigation Compose	11 rotas: as 9 telas, Criar conta e Chamada em breve
Login	Firebase Authentication	E-mail/senha (Google fica para depois)
Dados	Cloud Firestore	Usuários, mensagens, documentos, consultas
Arquivos	Não usado nesta fase	Firebase Storage exige plano Blaze (cartão) desde 03/02/2026; ver seção 8
IA	Simulada (respostas fixas)	Substituída por uma IA real numa fase futura

Todo o projeto roda no plano gratuito (Spark) do Firebase, sem cartão de crédito.

Identificação: projeto no Firebase `goianinho-medicina` e nome do pacote `com.goianinho.medicina`, igual no Firebase e no IntelliJ (pacote Android não aceita hífen).

Configuração e versões da fundação:

Item	Valor	Observação
Projeto Firebase	`goianinho-medicina` (ID `goianinho-1cc1f`)	Membros do time com papel Editor
Authentication	E-mail/senha ativado	Usuário de teste `paciente.teste@goianinho.com`
Firestore	`southamerica-east1` (São Paulo), modo produção	Local não pode ser alterado
Firebase BoM	34.19.0	Define as versões de `firebase-auth` e `firebase-firestore`
Plugin google-services	4.5.0	Lê o `google-services.json` da pasta `app/`
Navigation Compose	2.9.8	A 2.10 exige compileSdk 37 e AGP 9.2; o projeto está no API 36
Lifecycle (ViewModel e runtime para Compose)	Mesma versão do lifecycle do projeto	Permite `viewModel()` e `collectAsStateWithLifecycle()`
Android	compileSdk 36 · minSdk 26 (Android 8.0) · JDK 21	Gradle em Kotlin DSL com catálogo `libs.versions.toml`

Fluxo de dados em cada tela:

```
flowchart LR
  S[Screen<br/>Composable] -->|evento| V[ViewModel]
  V -->|StateFlow| S
  V --> R[Repository]
  R --> F[(Firebase)]
```

A tela só desenha e repassa cliques. A ViewModel guarda o estado. O Repository é o único que fala com o Firebase.

Estrutura de pastas:

```
app/src/main/java/com/goianinho/medicina/
 ├── MainActivity.kt   (abre o tema e a navegação)
 ├── ui/
 |    ├── login/        (Módulo 1 — Login e Criar conta)
 |    ├── home/         (Módulo 1 — Início do paciente e do médico, barra de
4 abas)
 |    ├── chat/         (Módulo 2)
 |    ├── documentos/   (Módulo 3)
 |    ├── agenda/       (Módulo 4 — inclui Chamada em breve)
 |    ├── comum/        (TelaEmConstrucao — compartilhado)
 |    └── theme/        (cores e tema — compartilhado)
 ├── data/
 |    ├── model/        (5 classes de dados + StatusDocumento —
compartilhado)
 |    └── repository/   (acesso ao Firebase — cada módulo cria o seu)
 └── navigation/       (AppNavigation e objeto Rotas — compartilhado)
app/src/main/res/drawable/   (ícones de linha do protótipo: ic_home, ic_chat,
ic_documento, ic_revisao, ic_agenda, ic_send, ic_mic)
```

Cada pasta de módulo tem um `doc.md` com o recorte desta documentação para aquele módulo: arquitetura obrigatória, regras do repositório, telas, requisitos, código de referência e aulas do Google. Na raiz do repositório, o `README.md` apresenta o projeto a quem visita o GitHub. Hoje cada tela existe como **tela provisória** com os parâmetros de navegação definitivos; o integrante substitui o conteúdo pela tela real.

Identidade visual (do protótipo): fundo `#F4F7F6`, cor principal `#0F6E6A`, texto `#10302E`, texto secundário `#4A5F5D`, destaque do médico `#10302E`. Fonte Plus Jakarta Sans no protótipo; no app, a fonte padrão do Android por enquanto. Sem modo escuro e sem cor dinâmica: o app segue sempre as cores do protótipo.

5. Organização do time

Equipe de desenvolvimento e testes: Felipe Caetano, Antony Gabriel, João Victor de Sousa e Dhyego Ferreira dos Reis (líder do projeto e arquitetura). Os quatro programam e também formam a equipe de testes.

Cada integrante é dono de um módulo e trabalha numa branch própria, já criada no repositório. Tudo entra na `main` por Pull Request.

Módulo	Responsável	Branch	Visão do paciente	Visão do médico	Pasta
1 · Login e Início	[INTEGRANTE 1]	`feature/login`	Login (compartilhado) e Início do paciente	Início do médico	`ui/login`, `ui/home`
2 · Chat com o Goianinho	[INTEGRANTE 2]	`feature/chat-ia`	Chat com o Goianinho	Goianinho do médico	`ui/chat`
3 · Documentos	[INTEGRANTE 3]	`feature/documentos`	Documentos	Revisão de resumos	`ui/documentos`
4 · Agenda	[INTEGRANTE 4]	`feature/agenda`	Agenda	Agenda do dia	`ui/agenda`

Regra: cada módulo tem duas visões, e as duas são do mesmo dono. Quem pega um módulo entrega a tela do **paciente** e a tela do **médico** daquele módulo, com as ViewModels e o acesso aos dados de cada uma. Não existe "alguém faz só o lado do médico". Um módulo só é considerado pronto quando as duas visões passam nos critérios de aceite da sua seção.

Fluxo de trabalho (GitHub Flow):

1. O líder sobe a fundação na `main`: projeto criado, Firebase conectado, tema, navegação com as 9 rotas vazias, pastas e docs.
2. O líder cria as 4 branches a partir da `main`. Cada integrante clona o repositório e entra **somente na sua branch**.
3. Antes de começar o dia: conferir a branch com `git branch --show-current` e trazer as novidades com `git pull origin main`.
4. Commits pequenos e frequentes, no padrão `feat(agenda): lista de horários` ou `fix(chat): mensagem duplicada`, enviados com `git push origin <sua-branch>`.
5. Terminou uma etapa, abre Pull Request da sua branch para a `main`. Não espera o módulo inteiro.
6. **Somente o líder** revisa, aprova e faz o merge ("Squash and merge"). O integrante nunca clica em "Merge".

Regras:

- **Cada integrante trabalha somente na branch designada a ele.** Commit e push só na própria branch; nunca na branch de outro integrante.
- **Ninguém além do líder altera a `main`, de maneira alguma:** nada de commit ou push direto, merge pelo terminal ou pelo botão do GitHub, nem `git push --force`.
- Toda mudança chega à `main` por Pull Request, revisado e aprovado pelo líder.

- Arquivos compartilhados (`theme/` , `navigation/` , `data/model/` , `build.gradle`) só mudam com aviso no grupo e decisão do líder.
- Cada `doc.md` de módulo repete essas regras, com os comandos de Git já preenchidos para a branch daquele módulo.
- **Bloqueio técnico:** no plano gratuito do GitHub, a proteção de branch só funciona em repositórios **públicos**. Para o GitHub impedir de fato alterações na `main` de um repositório privado, o dono precisa do GitHub Pro, que é gratuito para estudantes pelo GitHub Student Developer Pack. Sem isso, a regra vale por combinado do time.
- **`google-services.json` fica no repositório (decisão mantida).** O arquivo não é uma senha: identifica o projeto no Firebase e vai dentro de todo APK instalado. Um `.env` não se aplica a apps Android, porque o plugin do Google precisa do arquivo na pasta `app/` para compilar. A proteção real dos dados vem das regras do Firestore (seção 10).

Etapas de entrega (datas das próximas etapas a definir pelo time):

Etapa	Entrega	Quem	Situação
0	Fundação na `main`	Líder	✅ Concluída
1	Telas visuais (só layout, dados fixos) de cada módulo	Todos	⏳ Próxima
2	Integração com Firebase (Auth e Firestore)	Todos	A fazer
3	Navegação ponta a ponta e testes entre módulos	Todos	A fazer
4	Ajustes finais e demonstração	Todos	A fazer

O Login entra primeiro na `main` , porque os outros módulos dependem de usuário logado. Até lá, cada um usa um usuário fixo de teste.

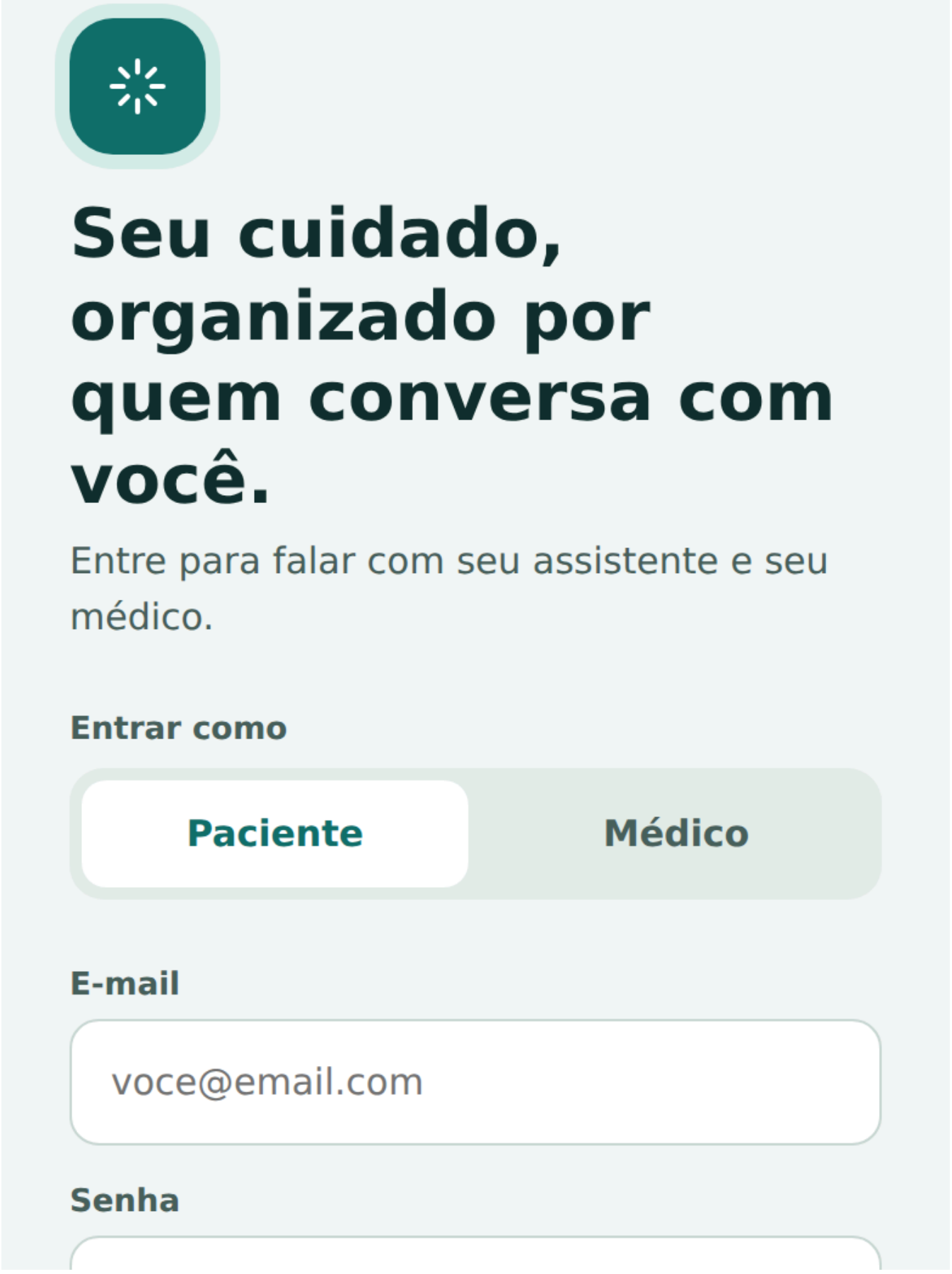
O que a fundação (Etapa 0) entregou:

- [x] ~~Projeto Firebase criado, app Android registrado, Authentication e Firestore ativos, regras de "só logado" publicadas e conexão testada no celular~~
- [x] ~~Projeto Kotlin + Compose no IntelliJ IDEA, com Firebase e as bibliotecas de navegação e lifecycle no Gradle~~
- [x] ~~Tema com as cores do protótipo, as 5 classes de dados e as constantes de status~~
- [x] ~~Navegação com as 11 rotas, barra de 4 abas para cada perfil e telas provisórias com os parâmetros definitivos~~
- [x] ~~`doc.md` em cada pasta de módulo, `README.md` na raiz e `.gitignore` revisado~~
- [x] ~~Fundação enviada à `main` e as 4 branches criadas a partir dela~~
- [x] ~~Integrantes convidados no GitHub e no Firebase~~

6. Módulo 1 — Login e Início

Porta de entrada do app: identifica o usuário, manda para o fluxo de paciente ou de médico e abre o **Início** de cada perfil, o painel que resume o que importa e leva às outras abas. Telas de referência: **1 · Login**, **1 · Início do paciente** e **1 · Início do médico**.

Elementos da tela Login: seletor Paciente/Médico, campos de e-mail e senha, link "Esqueci minha senha", botão Entrar, botão "Continuar com Google" e link "Criar conta".


Seu cuidado, organizado por quem conversa com você.
Entre para falar com seu assistente e seu médico.
Entrar como
Paciente
Médico
E-mail
voce@email.com
Senha

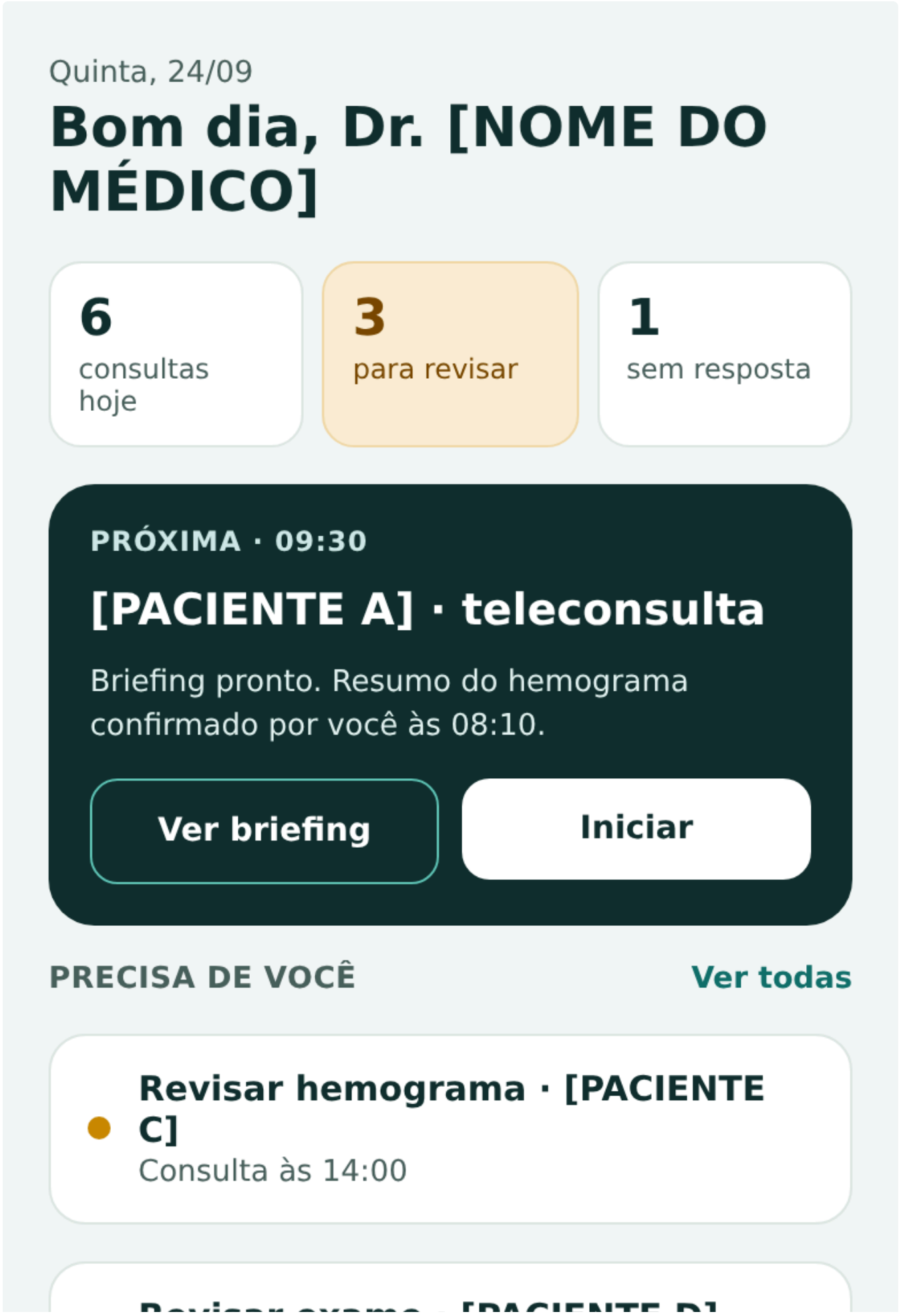


Elementos da tela Início (paciente): saudação com o nome e a data, card do Goianinho com campo "Fale ou escreva" e botão de voz que abrem o Chat, chips Marcar consulta / Enviar exame / Tirar dúvida, card da próxima consulta e lista "Acompanhamento" com o status dos documentos.


Quarta, 23/09
Olá, [NOME DO PACIENTE]
Em que posso ajudar hoje?
Goianinho · assistente do Dr. [NOME DO MÉDICO]
Fale ou escreva para o Goianinho...
Marcar consulta
Enviar exame
Tirar dúvida
Qui
24
PRÓXIMA CONSULTA
Dr. [NOME DO MÉDICO]
Ver
Teleconsulta · 09:30
ACOMPANHAMENTO
Hemograma completo
Resumo pronto · aguardando o médico



Elementos da tela Início (médico): saudação com a data, três contadores do dia (consultas, para revisar, sem resposta), card da próxima consulta com briefing e botões Ver briefing / Iniciar, lista "Precisa de você" com revisões pendentes e pacientes sem resposta, e campo "Pergunte ao Goianinho".


Quinta, 24/09
Bom dia, Dr. [NOME DO MÉDICO]
6
consultas hoje
3
para revisar
1
sem resposta
PRÓXIMA · 09:30
[PACIENTE A] · teleconsulta
Briefing pronto. Resumo do hemograma confirmado por você às 08:10.
Ver briefing
Iniciar
PRECISA DE VOCÊ
Ver todas
Revisar hemograma · [PACIENTE C]
Consulta às 14:00



Requisitos funcionais (prioridade: Must = obrigatório, Should = importante, Could = se sobrar tempo)

ID	Requisito	Prioridade
RF-01	O app deve permitir login com e-mail e senha via Firebase Auth.	Must
RF-02	O app deve permitir escolher o perfil (Paciente ou Médico) antes de entrar.	Must
RF-03	Após o login, o app deve abrir o Início do paciente ou o Início do médico, conforme o perfil salvo.	Must
RF-04	O app deve permitir criar conta com nome, e-mail, senha e perfil.	Must
RF-05	O app deve permitir recuperar senha por e-mail.	Should
RF-06	O app deve permitir login com conta Google.	Could
RF-07	O app deve manter o usuário logado ao reabrir.	Should

ID	Requisito	Prioridade
RF-31	O Início do paciente deve mostrar saudação com o nome, o atalho para conversar com o Goianinho e os chips Marcar consulta, Enviar exame e Tirar dúvida.	Must
RF-32	O Início do paciente deve mostrar a próxima consulta e o status dos documentos (dados dos Módulos 3 e 4).	Must
RF-33	O Início do médico deve mostrar os três contadores do dia: consultas, resumos para revisar e pacientes sem resposta.	Must
RF-34	O Início do médico deve mostrar a próxima consulta com briefing e a lista "Precisa de você".	Must
RF-35	Cada card e chip do Início deve levar à aba correspondente.	Should
RF-36	Os dois perfis devem ter barra inferior com quatro abas, começando pelo Início.	Must

Regras de negócio

- RN-01: o perfil fica salvo no documento do usuário no Firestore. Na próxima entrada, vale o perfil salvo, não o seletor.
- RN-02: senha com no mínimo 6 caracteres (limite do Firebase Auth).
- RN-10: o Início só **lê** dados. Quem grava consultas e documentos são os Módulos 3 e 4; o Início apenas resume e leva até eles.

Dados: coleção `usuarios` (dono deste módulo) e leitura de `consultas` e `documentos` para montar o Início (ver seção 10).

Critérios de aceite

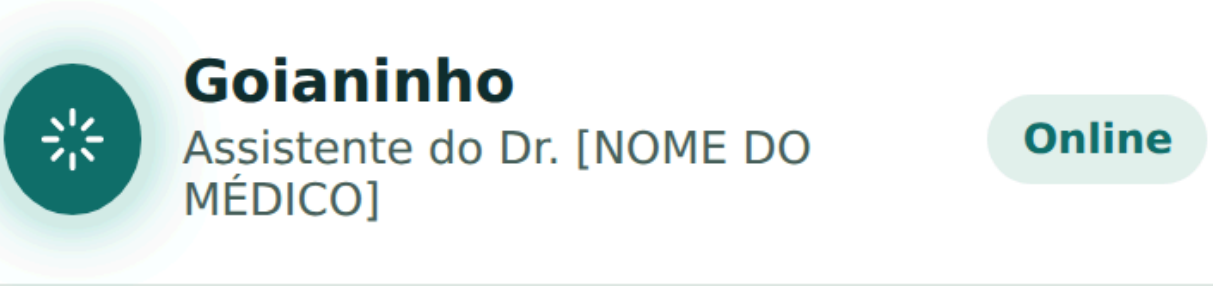
- [ ] Login com dados corretos abre o Início certo para cada perfil
- [ ] Senha errada mostra mensagem clara, sem fechar o app
- [ ] Conta criada aparece no Firebase Auth e em `usuarios`
- [ ] Fechar e reabrir o app mantém o usuário logado
- [ ] Início do paciente mostra o nome, a próxima consulta e os documentos reais do usuário logado
- [ ] Início do médico mostra contadores e lista "Precisa de você" com dados reais
- [ ] Barra de quatro abas funciona nos dois perfis

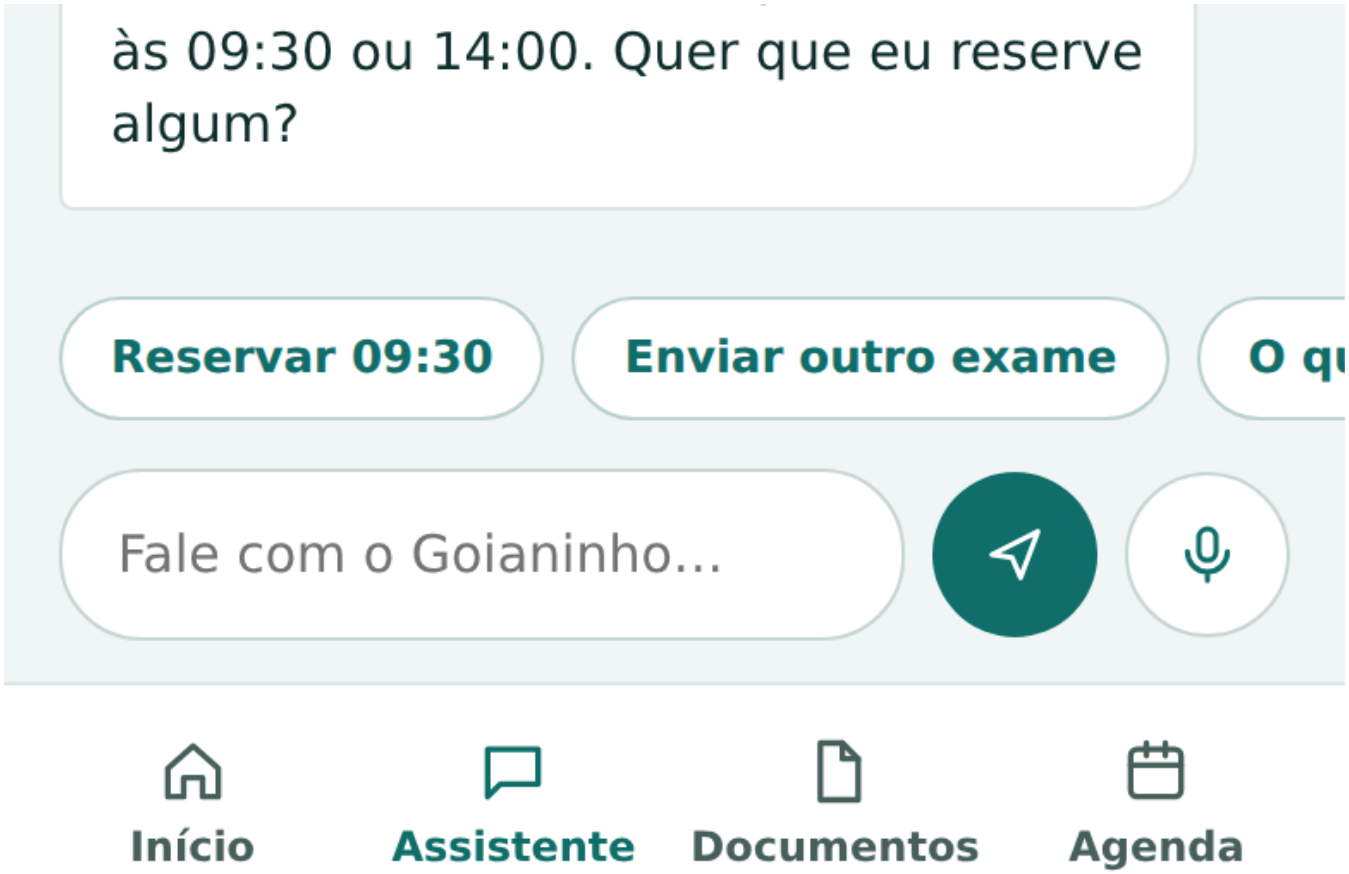
- [ ] Visual das três telas igual ao protótipo

7. Módulo 2 — Chat com o Goianinho

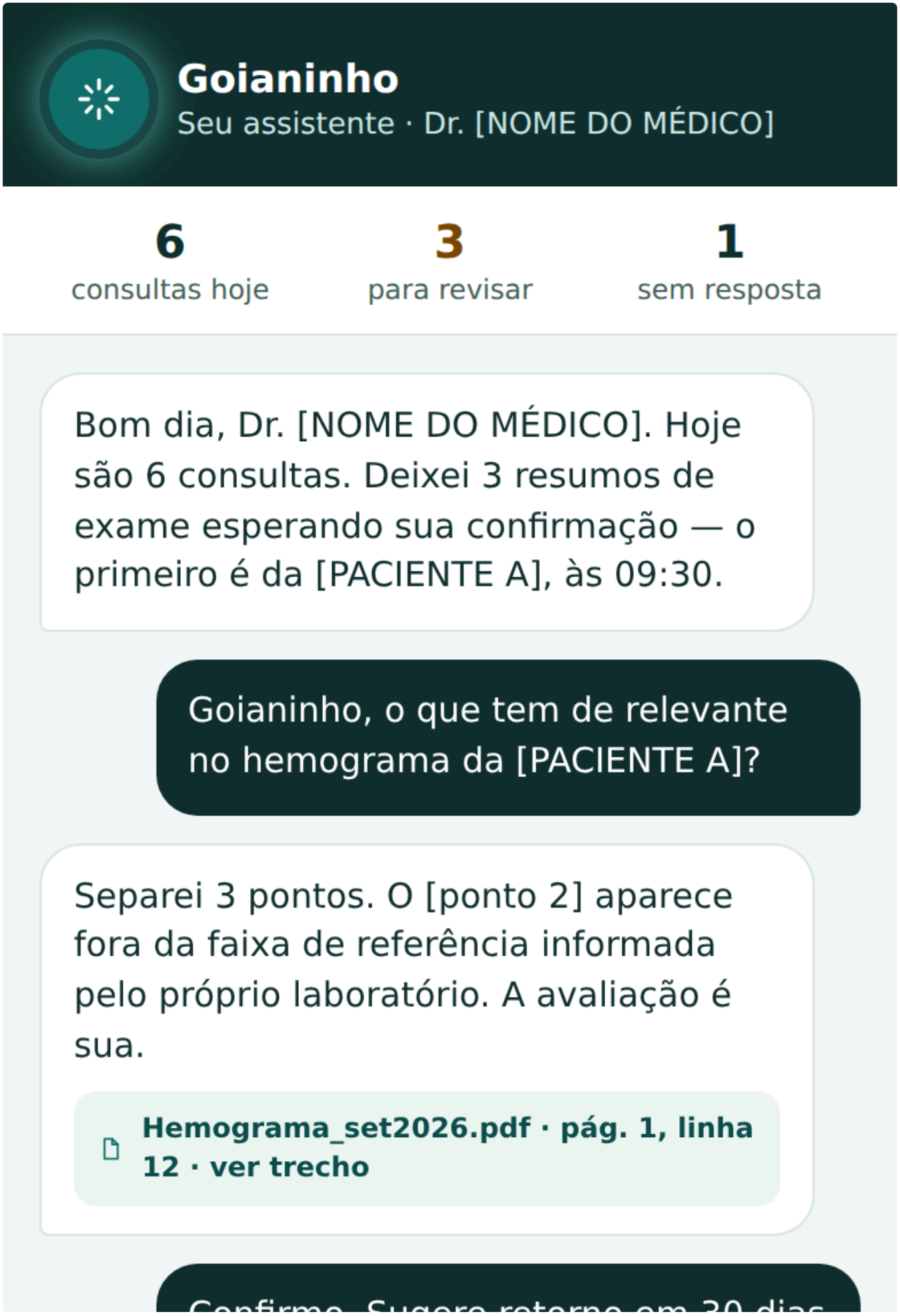
É o coração do projeto: a conversa em linguagem natural com o assistente, para paciente e médico. Telas de referência: **2 · Chat com o Goianinho** (paciente) e **2 · Goianinho do médico**.

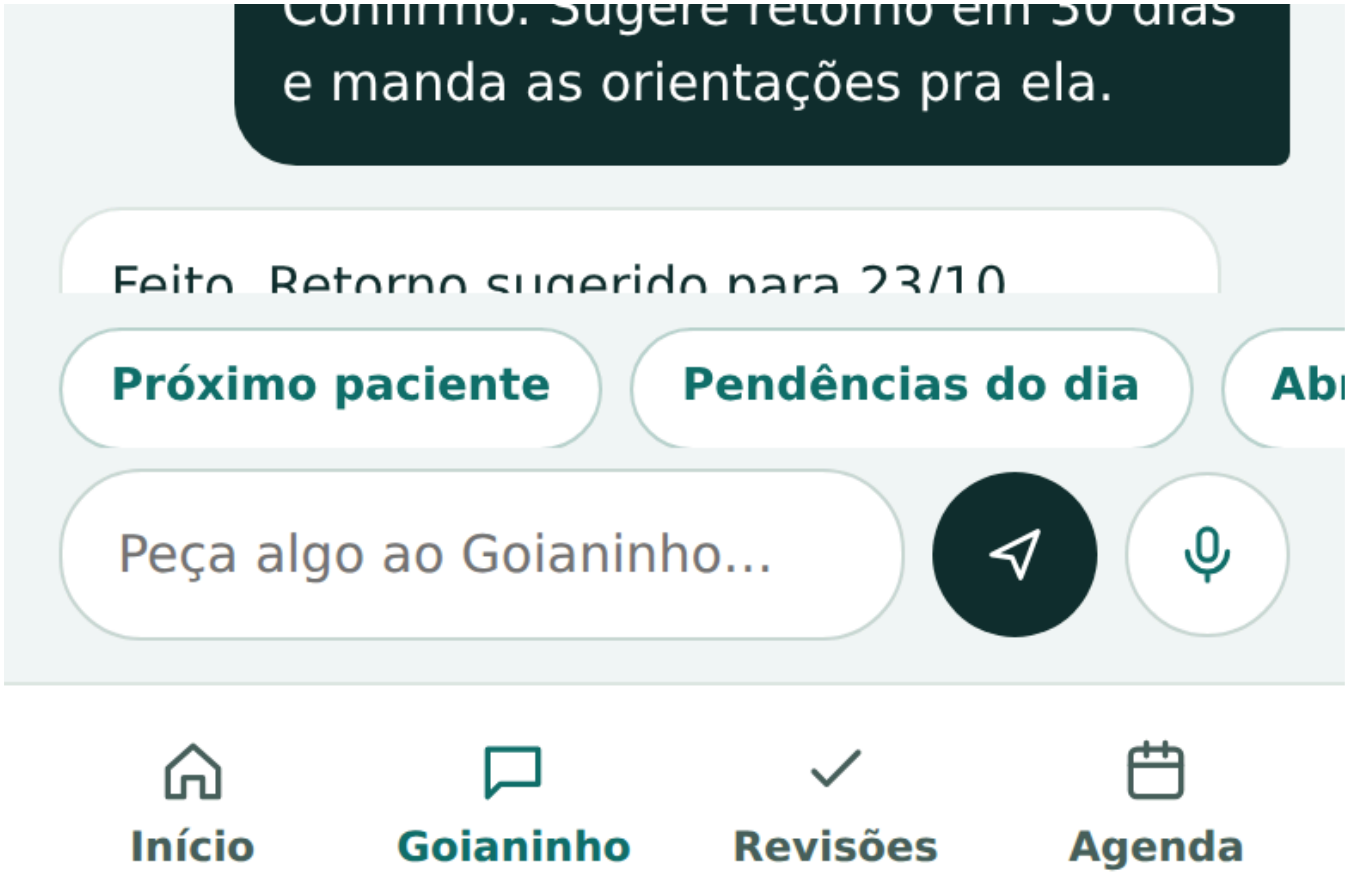
Elementos da tela (paciente): cabeçalho com avatar do Goianinho e "Assistente do Dr. X", lista de mensagens em bolhas, card de fonte dentro da resposta da IA, chips de ação rápida, campo de texto, botão enviar e botão de voz.


Goianinho
Assistente do Dr. [NOME DO MÉDICO]
Online
Olá, [NOME DO PACIENTE]. Sou o Goianinho, assistente do Dr. [NOME DO MÉDICO]. Posso marcar consultas, organizar seus exames e levar tudo pronto para ele.
Goianinho, mandei meu exame de sangue. Pode apresentar pro médico e ver um horário essa semana?
Recebi o hemograma. Organizei os pontos principais num resumo e enviei para o Dr. [NOME DO MÉDICO] revisar — só ele confirma o resultado.
Fonte: Hemograma_set2026.pdf · pág. 1, linhas 8–14
Ele tem teleconsulta na quinta (24/09)



Diferenças na visão do médico: cabeçalho escuro, faixa com resumo do dia (consultas, revisões pendentes, pacientes sem resposta), fonte clicável que abre a Revisão, chips "Próximo paciente" e "Pendências do dia".


Goianinho
Seu assistente · Dr. [NOME DO MÉDICO]
6
consultas hoje
3
para revisar
1
sem resposta
Bom dia, Dr. [NOME DO MÉDICO]. Hoje são 6 consultas. Deixei 3 resumos de exame esperando sua confirmação — o primeiro é da [PACIENTE A], às 09:30.
Goianinho, o que tem de relevante no hemograma da [PACIENTE A]?
Separei 3 pontos. O [ponto 2] aparece fora da faixa de referência informada pelo próprio laboratório. A avaliação é sua.
Hemograma_set2026.pdf · pág. 1, linha 12 · ver trecho



Requisitos funcionais

ID	Requisito	Prioridade
RF-08	O app deve exibir o histórico de mensagens do usuário, salvo no Firestore.	Must
RF-09	O app deve enviar mensagem digitada e mostrar uma resposta do Goianinho.	Must
RF-10	Respostas do Goianinho que citam um documento devem mostrar o card de fonte (arquivo, página, linha).	Must
RF-11	Os chips de ação rápida devem enviar a frase correspondente como mensagem.	Must
RF-12	Na visão do médico, o app deve mostrar os três contadores do dia.	Should
RF-13	No médico, tocar na fonte deve abrir a tela de Revisão.	Should
RF-14	O botão de voz deve converter fala em texto (reconhecimento de voz do Android).	Could

Regras de negócio

- RN-03: o Goianinho nunca afirma diagnóstico. Toda resposta sobre exame termina lembrando que o médico confirma (P-01).
- RN-04: nesta fase, as respostas vêm de uma lista fixa de pares "frase-gatilho → resposta" dentro do Repository. Mensagem sem gatilho recebe a resposta padrão.

O que fica simulado: a inteligência do Goianinho. Deixe a geração de resposta isolada numa função (`gerarResposta(mensagem)`) para trocar por IA real depois sem mexer na tela.

Dados: coleção `mensagens` (ver seção 10).

Critérios de aceite

- [ ] Mensagem enviada aparece na hora e continua lá ao reabrir o app
- [ ] Cada chip gera a resposta esperada
- [ ] Card de fonte aparece só nas respostas que citam documento
- [ ] Paciente e médico veem cada um a sua versão da tela
- [ ] Visual igual ao protótipo

8. Módulo 3 — Documentos e Revisão

O paciente envia exames, o Goianinho gera um resumo com fontes, e o médico confirma ou corrige. Telas de referência: **3 · Documentos** (paciente) e **3 · Revisão de resumos** (médico).

Decisão (v1.1): sem Firebase Storage nesta fase. O Storage passou a exigir o plano Blaze, com cartão de crédito, para qualquer uso. Para manter o projeto 100% gratuito, o app **não guarda o arquivo**: grava no Firestore só os dados do documento (nome, tipo, data, status e resumo simulado). A tela continua igual ao protótipo. Foto pequena pode ser salva como texto Base64 no próprio documento, respeitando o limite de 1 MB do Firestore.

Elementos da tela (paciente): botões "Enviar arquivo" e "Tirar foto", card do documento mais recente com o "Resumo do Goianinho" (cada ponto com página e linha), etiqueta de status e lista de documentos anteriores.

Documentos

O Goianinho lê, resume e leva para o seu médico revisar.

Enviar arquivo | **Tirar foto**

Hemograma completo

Enviado hoje · PDF · 2 páginas

Aguardando médico

Resumo do Goianinho

[Ponto relevante 1 extraído do exame]

pág. 1, linha 9

[Ponto relevante 2 extraído do exame]

pág. 1, linha 12

[Ponto relevante 3 extraído do exame]

pág. 2, linha 3

Resumo organizado pela IA. O resultado só vale após confirmação do médico.

Perguntar ao Goianinho sobre este exame



Elementos da tela (médico): fila de pacientes com resumo pendente, card do documento com os pontos e o link "Ver no documento", campo de observação, botões Corrigir e Confirmar, e aviso de registro no prontuário.

Revisões

O Goianinho organiza. Você confirma ou corrige.

[PACIENTE A] · 09:30 | **[PACIENTE C] · 14:0**

Hemograma completo

Aguardando você

[PACIENTE A] · enviado hoje, 07:42

[Ponto relevante 1 extraído pela IA]

pág. 1, linha 9 **Ver no documento**

[Ponto relevante 2 — fora da referência do laboratório]

pág. 1, linha 12 **Ver no documento**

[Ponto relevante 3 extraído pela IA]

pág. 2, linha 3 **Ver no documento**

Sua observação (opcional)

Corrija ou complemente o resumo...



Requisitos funcionais

ID	Requisito	Prioridade
RF-15	O paciente deve selecionar um PDF ou imagem do aparelho; o app grava no Firestore o nome, o tipo e a data (o arquivo não é enviado).	Must
RF-16	O paciente deve tirar foto de um documento pela câmera; se a imagem comprimida tiver menos de 1 MB, pode ser salva em Base64 no Firestore.	Should
RF-17	O app deve listar os documentos do paciente, do mais recente ao mais antigo.	Must
RF-18	Cada documento deve exibir o resumo do Goianinho com a fonte de cada ponto.	Must
RF-19	Cada documento deve exibir um status: Aguardando médico, Confirmado, Corrigido ou Arquivado.	Must
RF-20	O médico deve ver a fila de documentos com status "Aguardando médico".	Must

ID	Requisito	Prioridade
RF-21	O médico deve confirmar ou corrigir um resumo, com observação opcional.	Must
RF-22	Ao confirmar, o app deve registrar data, médico e que houve apoio de IA.	Must
RF-23	"Ver no documento" deve mostrar a foto salva em Base64 ou, sem foto, os dados do documento.	Could

Regras de negócio

- RN-05: o resumo só aparece como válido para o paciente depois da confirmação do médico (P-01).
- RN-06: todo documento confirmado guarda `confirmadoPor`, `confirmadoEm` e `apoioIA: true` (Resolução CFM 2.454/2026).
- RN-07: um médico só vê documentos de pacientes vinculados a ele.

O que fica simulado: a leitura do exame e o armazenamento do arquivo. Ao selecionar um documento, o app grava um resumo fixo de exemplo com três pontos e fontes fictícias. Não usar valores clínicos reais.

Dados: coleção `documentos` no Firestore (ver seção 10). Nada no Storage.

Critérios de aceite

- [ ] Documento selecionado aparece no Firestore e na lista do paciente
- [ ] Documento novo chega na fila do médico como "Aguardando médico"
- [ ] Confirmar e Corrigir mudam o status nas duas telas
- [ ] Os campos de registro são gravados ao confirmar
- [ ] Nenhum documento do Firestore passa de 1 MB
- [ ] Visual igual ao protótipo

9. Módulo 4 — Agenda

O paciente marca e acompanha consultas; o médico vê o dia com um briefing pronto de cada paciente. Telas de referência: **4 · Agenda** (paciente) e **4 · Agenda do dia** (médico).

Elementos da tela (paciente): card da próxima consulta com botão "Entrar na chamada", seletor de dias, grade de horários, botão de confirmar que mostra a escolha, e dica para marcar falando com o Goianinho.

Agenda

PRÓXIMA CONSULTA

Dr. [NOME DO MÉDICO]

Teleconsulta · Qui, 24/09 · 09:30

Entrar na chamada

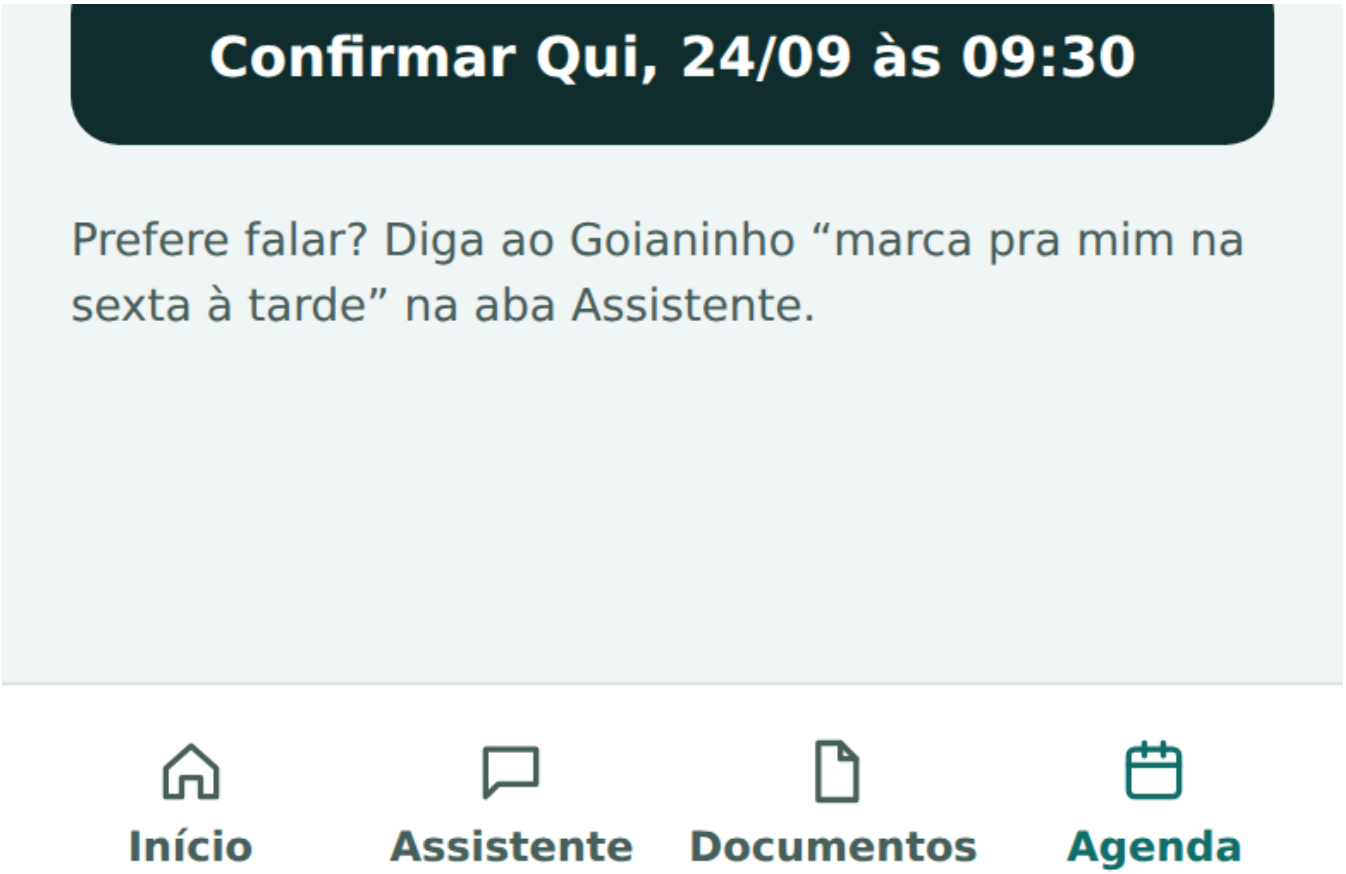
O Goianinho te lembra 1 hora antes pelo WhatsApp.

Marcar nova consulta

Qui	Sex	Seg	Ter
24	25	28	29

Horários disponíveis

08:00	09:30	11:00
14:00	15:30	17:00



Elementos da tela (médico): card do paciente atual com o briefing do Goianinho e os botões "Ver resumo" e "Iniciar", e lista das próximas consultas com status e dica de engajamento (ex.: "prefere ligação").

Agenda de hoje

Quinta, 24/09 · 6 consultas

AGORA · 09:30

[PACIENTE A]

Briefing do Goianinho

Motivo relatado: [MOTIVO DA CONSULTA]. Hemograma enviado e resumo confirmado por você às 08:10.

Ver resumo

Iniciar

PRÓXIMAS

10:30 [PACIENTE B]

Sem exames novos

Costuma confirmar tarde · lembrete já enviado

14:00 [PACIENTE C]



Requisitos funcionais

ID	Requisito	Prioridade
RF-24	O paciente deve ver os horários livres do médico por dia.	Must
RF-25	O paciente deve reservar um horário, que fica indisponível para os outros.	Must
RF-26	O app deve exibir a próxima consulta do paciente no topo da tela.	Must
RF-27	O médico deve ver as consultas do dia em ordem de horário.	Must
RF-28	Cada consulta do médico deve mostrar o status do resumo do paciente (vem do Módulo 3).	Should
RF-29	Cada consulta do médico deve mostrar uma dica de engajamento do paciente.	Could
RF-30	"Entrar na chamada" e "Iniciar" devem abrir a videochamada.	Could

Regras de negócio

- RN-08: um horário reservado não pode ser reservado de novo (usar transação do Firestore).
- RN-09: nesta fase, os horários livres do médico são cadastrados manualmente no Firestore.

O que fica simulado: a videochamada (o botão pode abrir uma tela "Chamada em breve") e as dicas de engajamento (texto fixo por paciente).

Dados: coleções `horarios` e `consultas` (ver seção 10).

Critérios de aceite

- [ ] Horário reservado some da grade para outros pacientes
- [ ] Consulta reservada aparece na Agenda do dia do médico
- [ ] Status do resumo bate com o que está no Módulo 3
- [ ] Visual igual ao protótipo

10. Modelo de dados (Firestore)

São cinco coleções, definidas na fundação e compartilhadas por todos os módulos. Nome de campo novo só entra por PR avisado ao time.

Coleção	Campos principais	Dono	Também usam
`usuarios`	`uid`, `nome`, `email`, `perfil` (paciente/medico), `medicoId` (para paciente), `canalPreferido`	Módulo 1	Todos
`mensagens`	`pacienteId`, `medicoId`, `autor` (usuario/goianinho), `texto`, `fonte` (opcional), `criadaEm`	Módulo 2	—
`documentos`	`pacienteId`, `medicoId`, `nome`, `tipo` (pdf/imagem), `enviadoEm`, `imagemBase64` (opcional, < 1 MB), `resumo` (lista de ponto + fonte), `status`, `observacao`, `confirmadoPor`, `confirmadoEm`, `apoioIA`	Módulo 3	Módulos 2 e 4
`horarios`	`medicoId`, `data`, `hora`, `livre`	Módulo 4	—
`consultas`	`pacienteId`, `medicoId`, `dataHora`, `tipo` (teleconsulta), `status`, `motivo`	Módulo 4	Módulo 2

```
flowchart LR
  U[usuarios] --> M[mensagens]
  U --> D[documentos]
  U --> C[consultas]
  H[horarios] --> C
  D -.status do resumo.-> C
```

Storage: não usado nesta fase (exige plano Blaze). Se for ativado no futuro, criar o bucket em US-CENTRAL1, US-EAST1 ou US-WEST1, as únicas regiões com cota gratuita.

Regras de segurança. Publicadas hoje (fase de desenvolvimento): só usuário logado lê ou grava qualquer documento. Isso ainda permite que um usuário logado veja dados de outros, então, **antes da apresentação**, as regras passam a ser por usuário: paciente só acessa o que tem o próprio `pacienteId`; médico só acessa o que tem o próprio `medicoId`.

11. Fora do escopo desta fase

Estes itens não devem ser desenvolvidos agora, para o time não se perder. Ficam como próximos passos do projeto.

Item	Por que fica para depois
IA real (LLM) respondendo no chat	Exige backend e custo de API; a tela já fica pronta para a troca
Leitura real de exames (OCR + resumo)	Mesma razão; depende da IA real
Upload real de arquivos (Firebase Storage)	Exige plano Blaze com cartão; hoje só os dados do documento vão para o Firestore
Videochamada funcionando	Exige SDK de vídeo; na fase atual o botão é simulado
Canal WhatsApp e ligação por voz	Parte da visão multicanal, depende de backend
Aprendizado real do perfil de engajamento	Precisa de histórico de uso; hoje as dicas são texto fixo
IA que sugere hipótese clínica	Muda o nível de risco regulatório; fica como visão de futuro

12. Glossário, referências e histórico

Termo	Significado
Goianinho	Nome do assistente de IA do app
RF	Requisito funcional: o que o app deve fazer
RN	Regra de negócio: restrição que o app deve respeitar
P	Princípio do produto (seção 2)
Must / Should / Could	Prioridade: obrigatório / importante / se sobrar tempo (método MoSCoW)
PR	Pull Request: pedido para juntar uma branch na `main`
MVVM	Padrão Model-View-ViewModel
doc.md	Guia de implementação que fica dentro da pasta de cada módulo
Tela provisória	Tela da fundação que já navega, mas ainda não tem o layout final; o dono do módulo substitui

Como este documento foi estruturado. Segue o modelo de especificação de requisitos da ISO/IEC/IEEE 29148 (sucessora da IEEE 830), com IDs únicos por requisito, prioridade MoSCoW e critérios de aceite verificáveis, simplificado para um time de 4 pessoas.

Referências

- Repositório do projeto (privado)
- Protótipo das telas
- Guide to app architecture (Android Developers) — base do MVVM adotado
- Android Basics with Compose and Firebase (Google) — curso de referência do time
- Documentação de Requisitos: escrita estruturada de cenários (Brasil Acadêmico, 2026) — modelo de ERS baseado na ISO/IEC/IEEE 29148
- Resolução CFM nº 2.454/2026 — uso de IA na medicina

Histórico de revisões

Versão	Data	Alteração
1.5	23/10/2026	Fundação concluída: configuração e versões registradas, estrutura de pastas final (11 rotas, ui/comum, ícones), etapas com situação, equipe nomeada, projeto identificado como da disciplina Softwares para Dispositivos Móveis, decisão sobre o google-services.json e estado atual das regras do Firestore
1.4	01/10/2026	Regras de branch: cada integrante trabalha só na branch designada; somente o líder altera a main (revisão, aprovação e merge de PR); nota sobre proteção de branch exigir GitHub Pro em repositório privado
1.3	01/10/2026	Telas de Início do paciente e do médico adicionadas ao Módulo 1 (RF-31 a RF-36, RN-10); barra inferior com 4 abas; regra explícita de que cada módulo entrega as visões do paciente e do médico; 9 telas no total
1.2	29/09/2026	Assistente renomeado de Íris para Goianinho (textos e imagens das telas); projeto goianinho-medicina e pacote com.goianinho.medicina definidos
1.1	27/09/2026	Firebase Storage removido desta fase (exige plano Blaze); Módulo 3 grava só dados no Firestore; IDE definida como IntelliJ IDEA
1.0	24/09/2026	Versão inicial: visão geral, organização do time e especificação dos 4 módulos

---

*Documento elaborado e arquitetado por **Dhyego Ferreira dos Reis**.*